

JEM Leasing — Phase 1 Setup Plan

Browser-based operations console (React + FastAPI) replacing the first slice of the Access front end.

What this is

Phase 1 of replacing the Access front end: a browser-based operations console (React) backed by a Python API (FastAPI), talking directly to your existing MySQL database. It reuses the AR/Sage sync data pipeline already built, so it's a working screen, not a "hello world."

What's in the project folder

```
erp-app/  
|-- backend/      FastAPI app - talks to MySQL, exposes /api/... endpoints  
|-- frontend/     React + TypeScript + Tailwind - the browser UI  
`-- sql/          (empty for now - migrations go here as the app grows)
```

Before you start: pick the right server

Do not put this on the same Windows Server 2008 box that runs Sage. That machine is 15+ years past its OS release and years past end of support - fine for a legacy accounting install nobody exposes to a browser, wrong foundation for a new app. Use a separate, currently-supported server (Windows Server 2019/2022, or Ubuntu Server 22.04/24.04), and set up two of these identically: one **test** VM, one **live** VM.

*Everything below assumes the **test** VM first.*

Step 1 — Install prerequisites on the test VM

If Ubuntu Server:

```
sudo apt update  
sudo apt install -y python3.11 python3.11-venv python3-pip git  
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt install -y nodejs  
sudo apt install -y mysql-client
```

If Windows Server: install Python 3.11+ (python.org, check "Add to PATH"), Node.js LTS (nodejs.org), Git (git-scm.com), and MySQL Workbench or the command-line client.

Step 2 — Create a test copy of the database

Never point new code at production first.

```
mysqldump -u root -p cash_test > backup.sql  
mysql -u root -p -e "CREATE DATABASE cash_test_dev"  
mysql -u root -p cash_test_dev < backup.sql
```

Create a dedicated MySQL user for the app (don't use root):

```
CREATE USER 'erp_app_user'@'%' IDENTIFIED BY 'choose-a-real-password';
GRANT SELECT, INSERT, UPDATE ON cash_test_dev.* TO 'erp_app_user'@'%';
FLUSH PRIVILEGES;
```

Step 3 — Get the code onto the test VM

Push the erp-app folder to a new GitHub repository from your dev machine, then on the test VM:

```
git clone <your-repo-url>
cd erp-app
```

Step 4 — Backend setup

```
cd backend
python3.11 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
```

Edit .env: point MYSQL_DATABASE at cash_test_dev, fill in the erp_app_user password from Step 2.

```
uvicorn app.main:app --reload --port 8000
```

Check it's alive: open <http://<test-vm-ip>:8000/api/health> in a browser - should show {"api":"ok","database":"ok"}.

Step 5 — Frontend setup

In a second terminal:

```
cd frontend
npm install
npm run dev
```

Open <http://<test-vm-ip>:5173> - you should see the operations console with four live panels: Sage sync status, pending exports, AR balances, and customer lookup.

If panels show "Couldn't load..." errors, it's almost always CORS or the wrong API URL - check frontend/.env (VITE_API_BASE=<http://<test-vm-ip>:8000>) and backend .env's CORS_ORIGINS.

Step 6 — Confirm it against what you know is true

Before trusting any of this: cross-check the AR balances and pending payments shown against what you already know from Access/Sage for a few real customers. Don't move to Step 7 until the numbers agree.

Step 7 — Commit and tag

```
git add .
git commit -m "Phase 1: operations console (sync status, AR, customer lookup)"
git tag v0.1.0
git push origin main --tags
```

Step 8 — Promote to the live VM (only after Step 6 passes)

```
# Always back up production first
mysqldump -u root -p cash_test > backup_before_v0.1.0.sql
```

```
git clone <your-repo-url> # or git pull if already cloned
cd erp-app/backend
git checkout v0.1.0
python3.11 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # edit: point at REAL production database this time
```

Run the backend as a persistent service rather than a terminal you can close: **Linux** - a systemd unit running `uvicorn app.main:app --port 8000`. **Windows** - NSSM (Non-Sucking Service Manager) wrapping the same command.

Build the frontend for production instead of the dev server:

```
cd frontend
npm install
npm run build
```

Serve the resulting `dist/` folder with any static file server (nginx, IIS, or the `serve` package) rather than `npm run dev`.

Rollback if anything looks wrong: `git checkout <previous-tag>`, restore `backup_before_v0.1.0.sql` if data was affected, restart the service.

What's next (Phase 2+)

- Add authentication (simple username/password + sessions is enough at this size)
- Add a real "post payment" screen backed by `dbo_payments` (write side)
- Start replacing individual Access modules one at a time, reusing this same backend/frontend project rather than starting fresh each time